

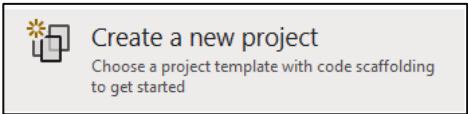
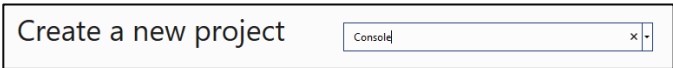
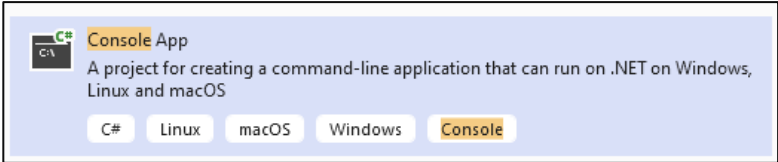
C# The Programming Language Part 1

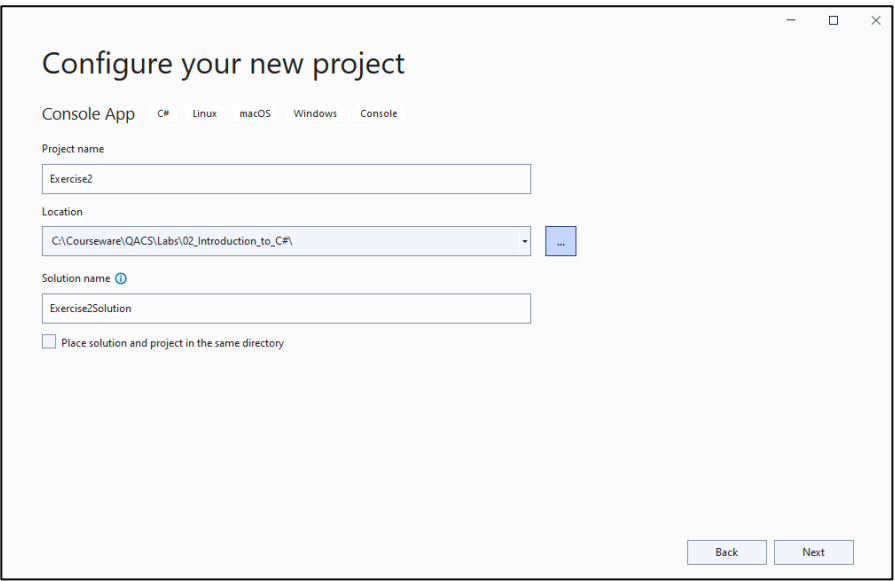
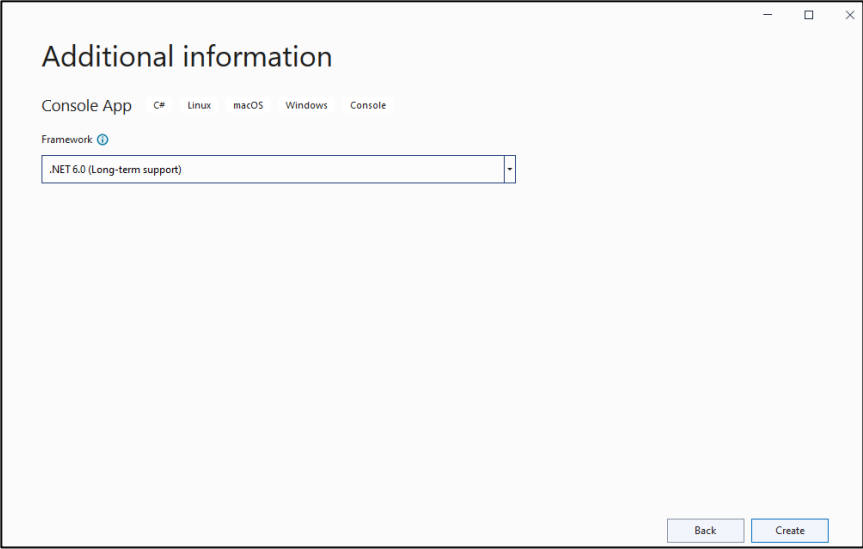
Exercise Workbook



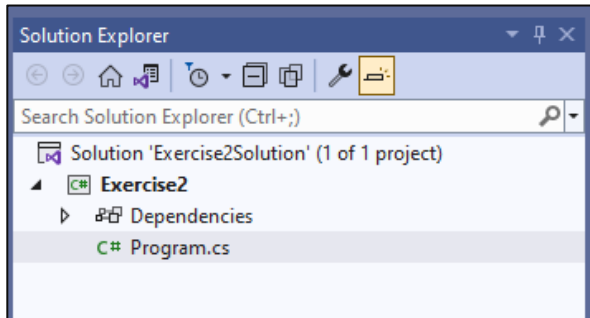
Introduction to C#

The objective of this exercise is to get you started using C# and the Visual Studio IDE and introduce you to simple debugging. You will also work with a test project and write some simple tests.

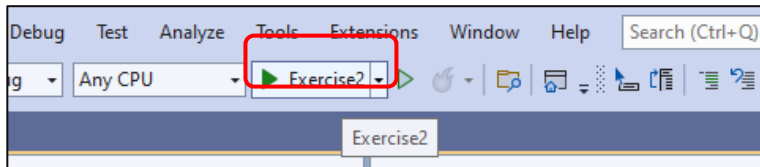
1	Start Microsoft Visual Studio 2026
2	Choose ' Create a new project ' 
3	In the Search box, type Console 
4	Select Console App and choose Next 
5	Name the <i>Project</i> Exercise2 and the <i>Solution</i> Exercise2Solution . Save the files in \Labs\02_Introduction_to_C#\Begin

	
6	Ensure .NET 10.0 (Long-term support) (or whatever version of .NET you are working with) is selected in Additional information and click Create 
7	You will see a code editor window for a file called Program.cs containing the following code: 

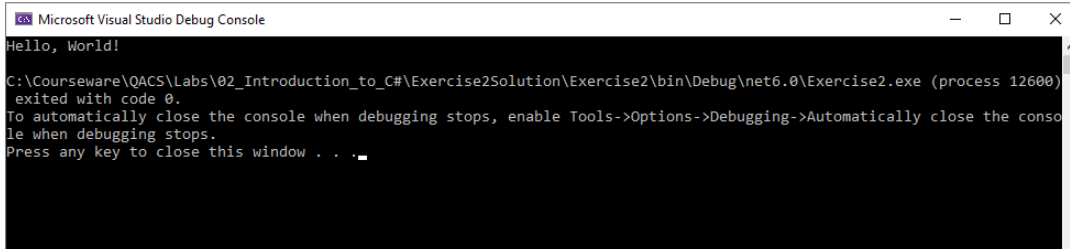
- 8 You can see the Solution Explorer window with **Program.cs** being tracked as the active file.



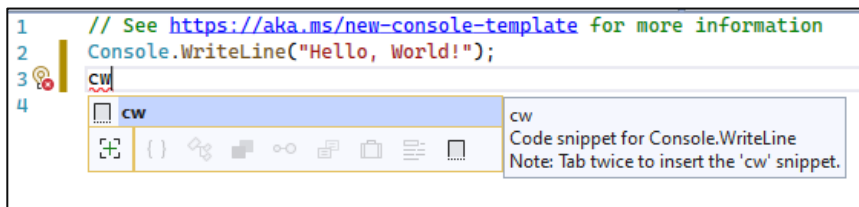
- 9 From the toolbar, click the green arrow to **Start with Debugging**:



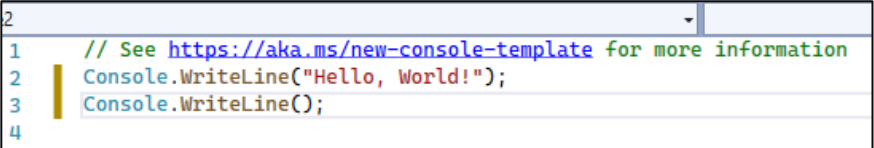
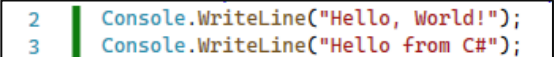
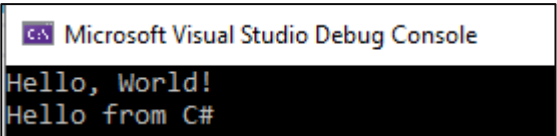
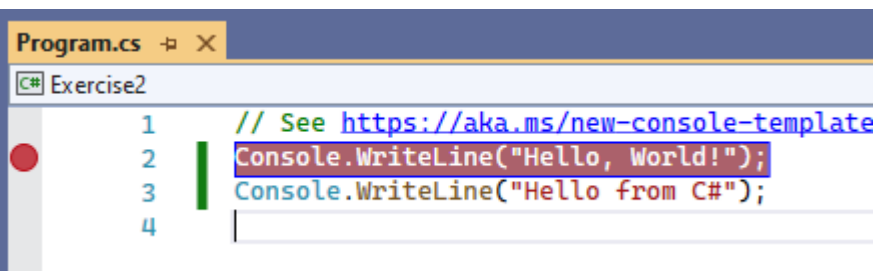
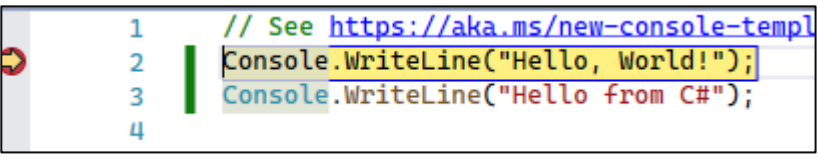
- 10 Observe the output of the program and press any key to close the console window.



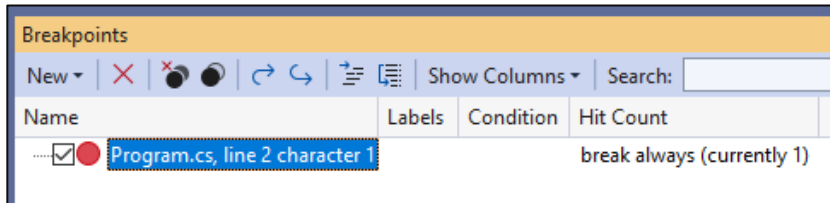
- 11 Add a line of code at line 3 as follows. Type:
CW
Notice it is a recognised code snippet for **Console.WriteLine**:



Tab twice to insert the code:

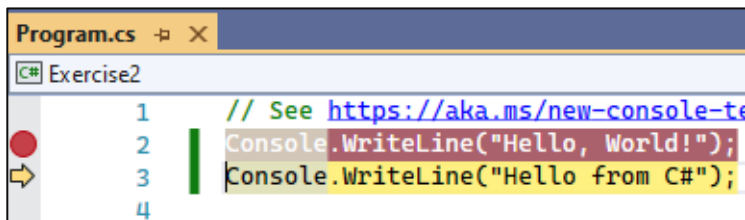
	 <pre>2 1 // See https://aka.ms/new-console-template for more information 2 Console.WriteLine("Hello, World!"); 3 Console.WriteLine(); 4</pre>
12	Add the text string "Hello from C#" as a parameter to the WriteLine method.  <pre>2 Console.WriteLine("Hello, World!"); 3 Console.WriteLine("Hello from C#");</pre>
13	Run the program again, this time using the keyboard shortcut F5  Press any key to quit the program.
14	You will now do some very simple debugging. Click in the margin well to the left of line 2 to set a breakpoint: 
15	Debug the program with F5 and notice how the application is now paused on your breakpoint:  Observe the changes to the Visual Studio layout. Numerous debug windows are now open including Autos, Locals, Watch 1, Call Stack, and Breakpoints.

- 16 Open the **Breakpoints** window and observe the **Hit Count** value is set to “**break always (currently 1)**”

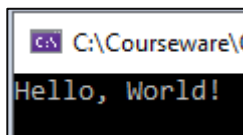


- 17 You will Step Into the next line of code which will run the line that is currently highlighted.

Press **F11**.

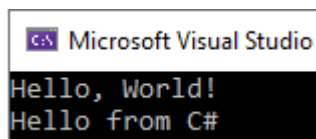


Look at the **Console** output window. You can see line 2 has run:



Press **F11** again.

The program ends because the last line of code has been run successfully. You can see the complete result in the output console:



If You Have Time – Using Git and GitHub

Use these instructions to familiarize yourself with GitHub (if you have not used GitHub previously)

Objective

Gain an understanding of how GitHub can be used to work on projects in a collaborative environment. You are encouraged to use GitHub as a repository for **ALL** the code you work on as part of this course. Using it means you are less likely to be impacted on a virtual machine timing out at some point. This is especially true of LOD machines and less so if you are using GoToMyPC. However, **ALL** VM's will be reset at the course's conclusion so **backing things up is a really good idea**.

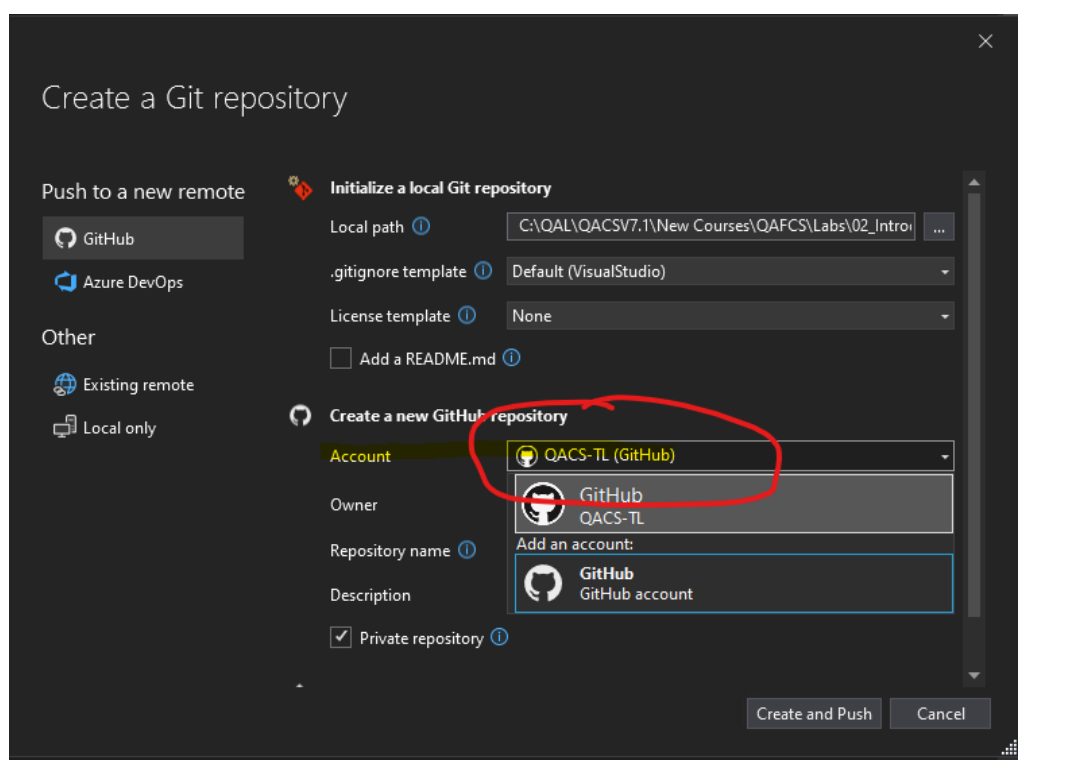
Requirements

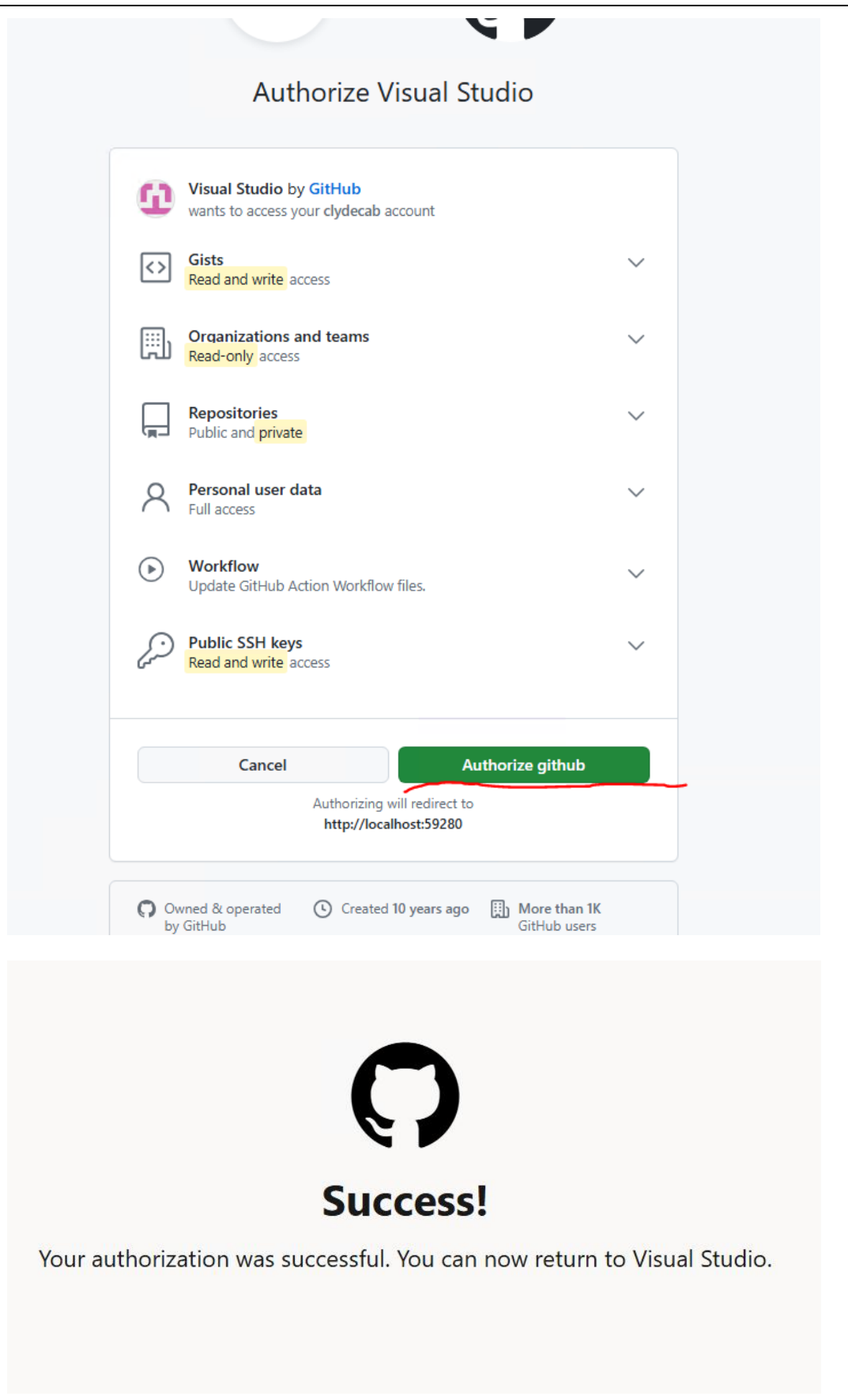
Create a GitHub account (if necessary) and then follow a the "hello world intro to GitHub" on the GitHub portal. Then use Visual Studio in conjunction with GitHub to manage versioning.

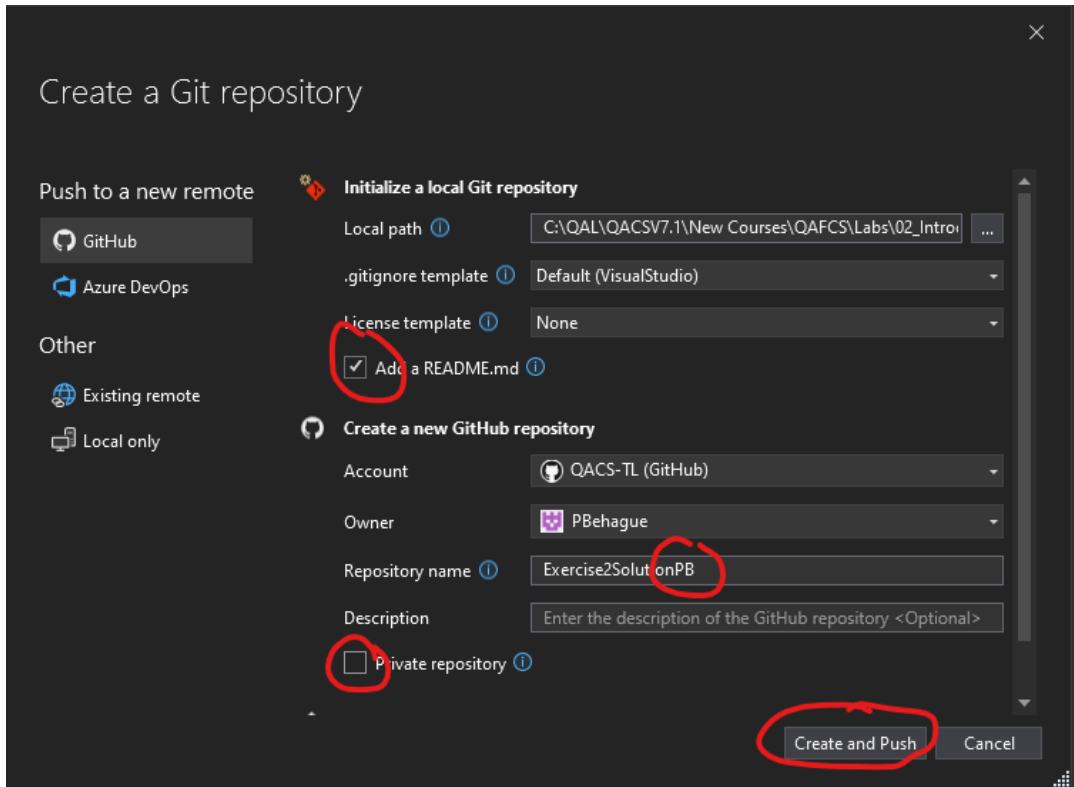
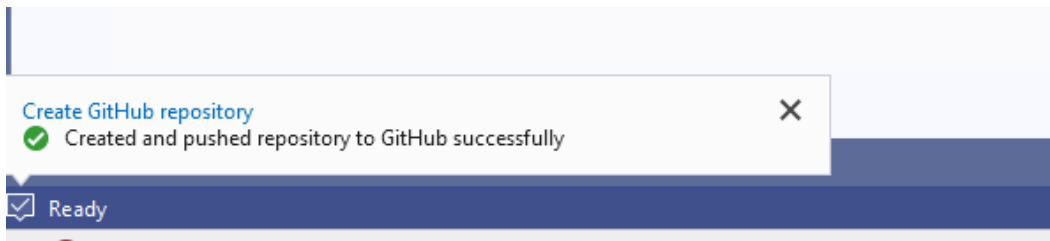
Steps to Complete

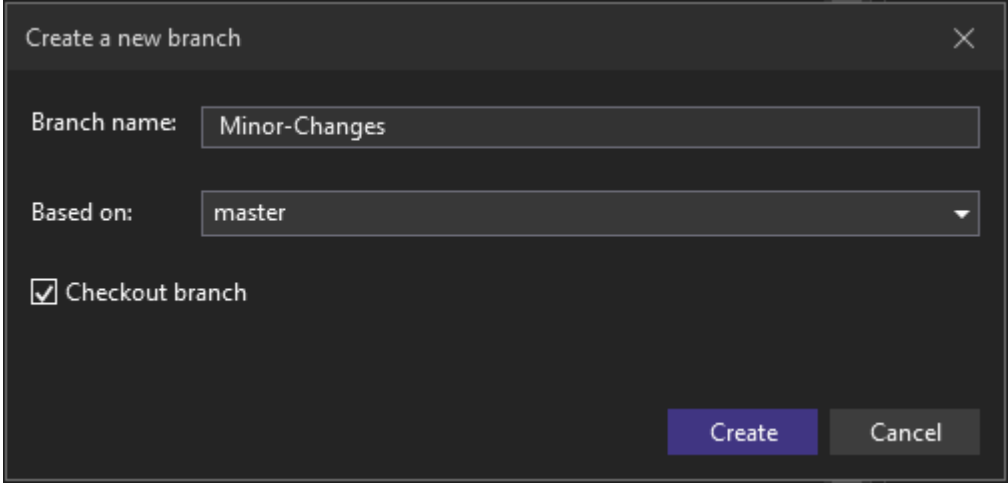
1	Create your GitHub account https://docs.github.com/en/get-started/start-your-journey/creating-an-account-on-github
2	Create a repository and use it to merge changes into the main branch from a another https://docs.github.com/en/get-started/start-your-journey/hello-world
3	Using GitHub with Visual Studio - Using Visual Studio Open the Solution called Labs\02_Introduction_to_C#\Begin - Under the Git item on the main menu select Create Git Repository - Sign in to GitHub using the account you used earlier.

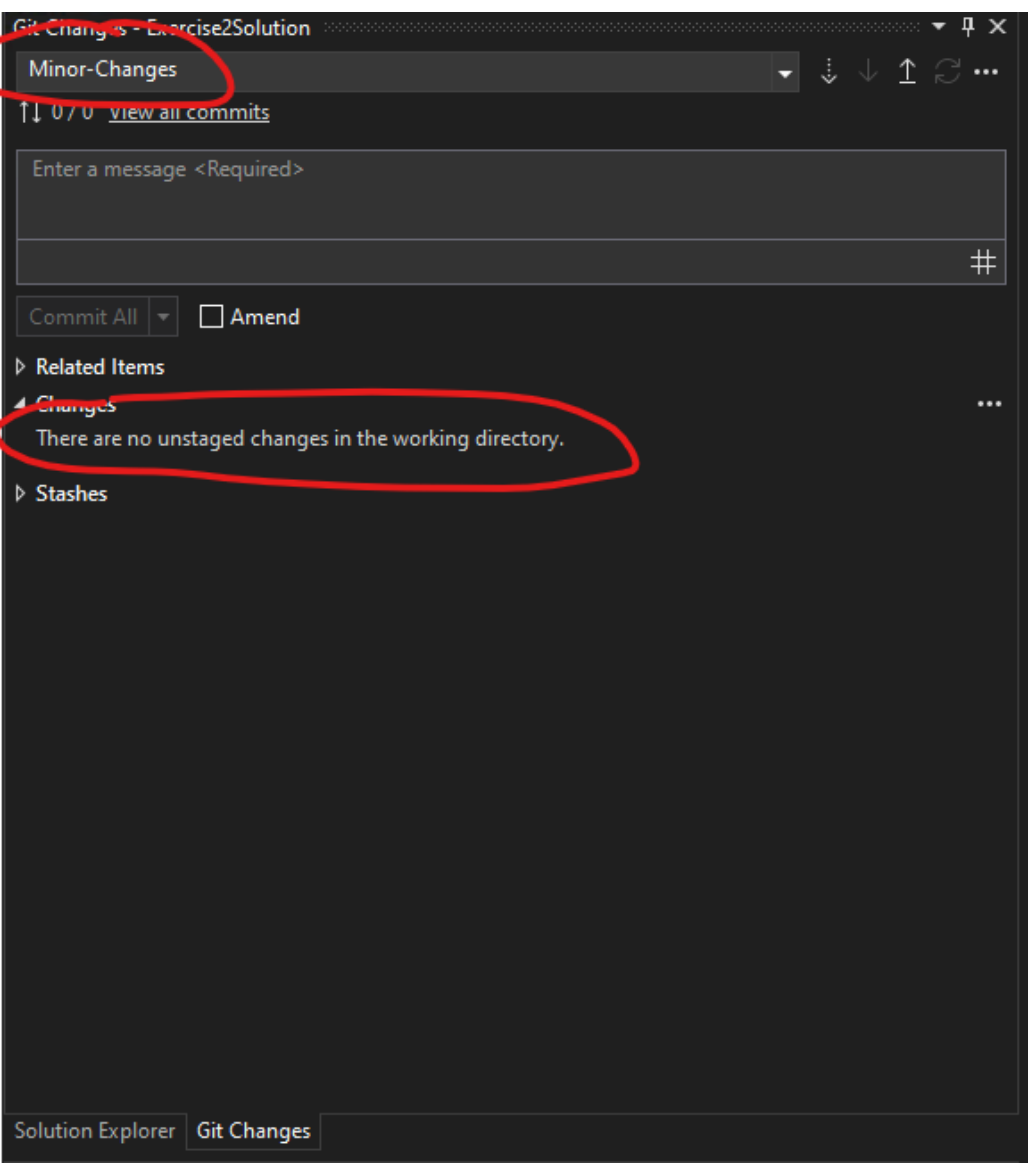
4

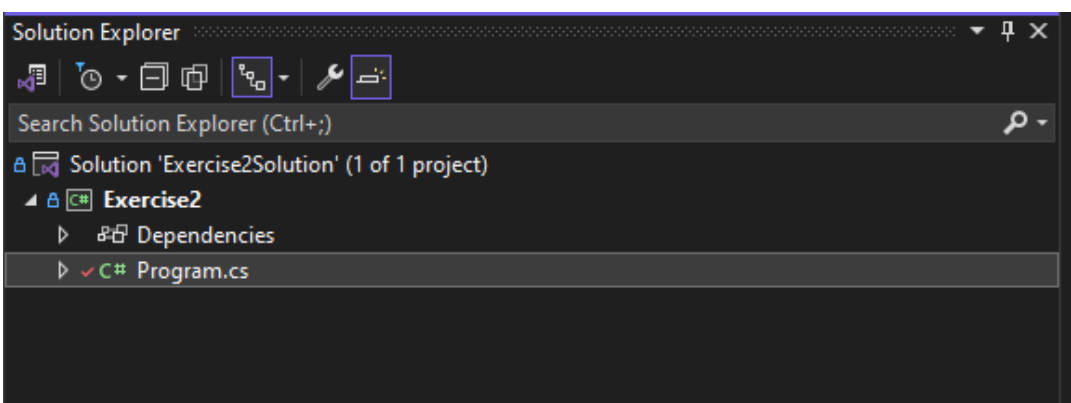
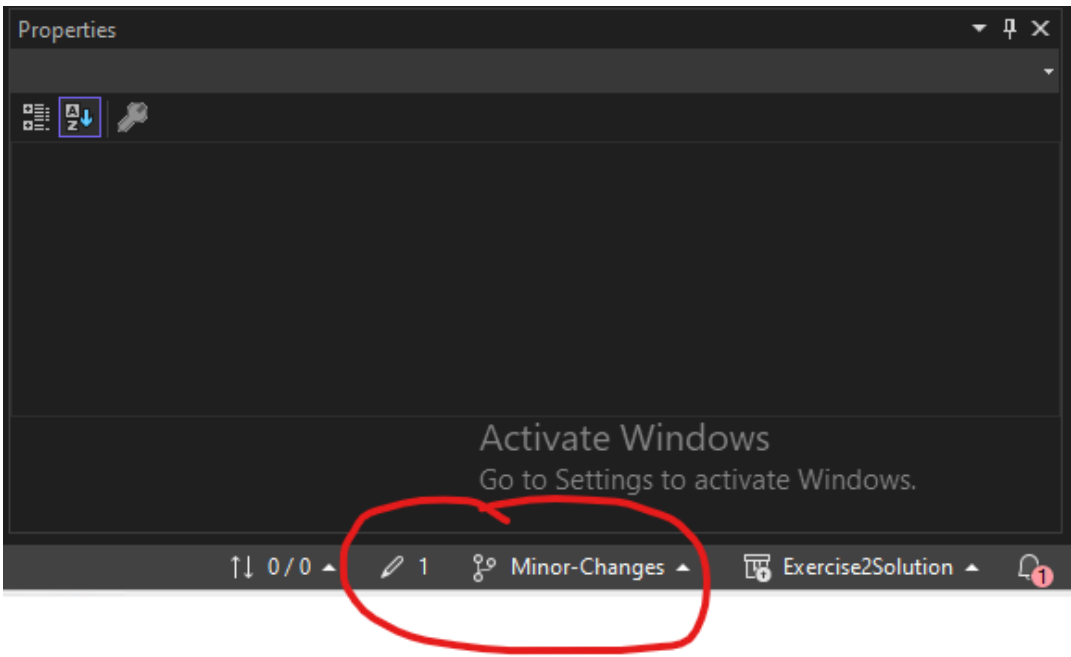




5	Add your initials to the end of the auto-generated repository name ("PB" in the example below), deselect the Private repository option and select the Add a README.md
6	
7	Then click on the Create and Push button
8	Your project has now been uploaded to GitHub and you also have a local copy(clone) on the local file system.
9	

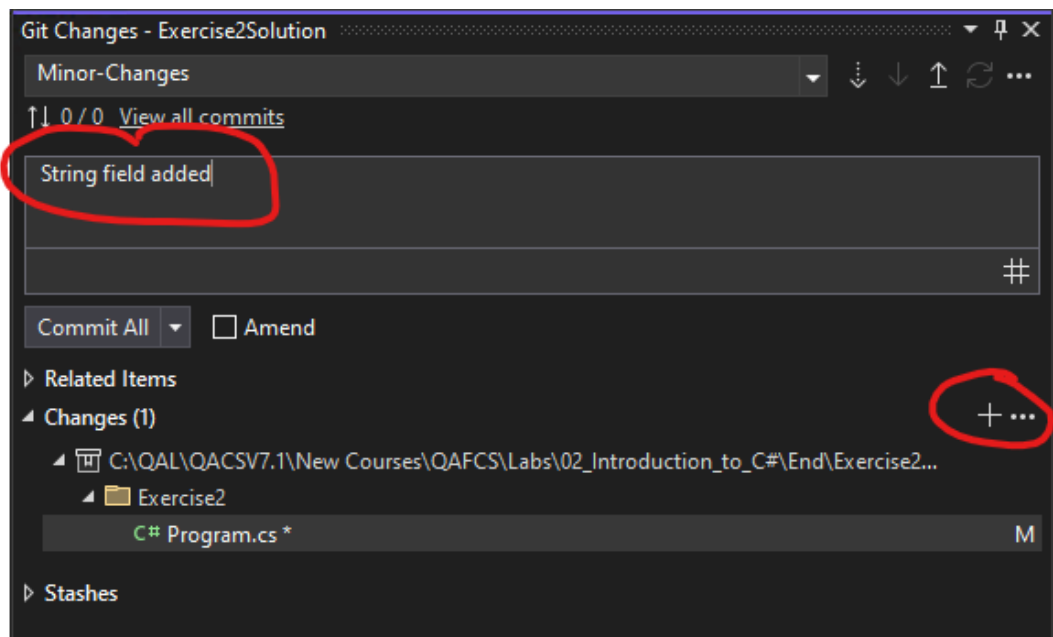
10	Now make a branch to support updates to the current project. Click on the Git item in the main menu of Visual Studio and select the New Branch Item. Name the branch Minor-Changes and confirm that the Checkout branch option is selected
11	
12	Click on create, and if prompted for further input, accept the defaults and continue.
13	The Git Changes window should show that there are currently no changes deployed to the branch named Minor-Changes

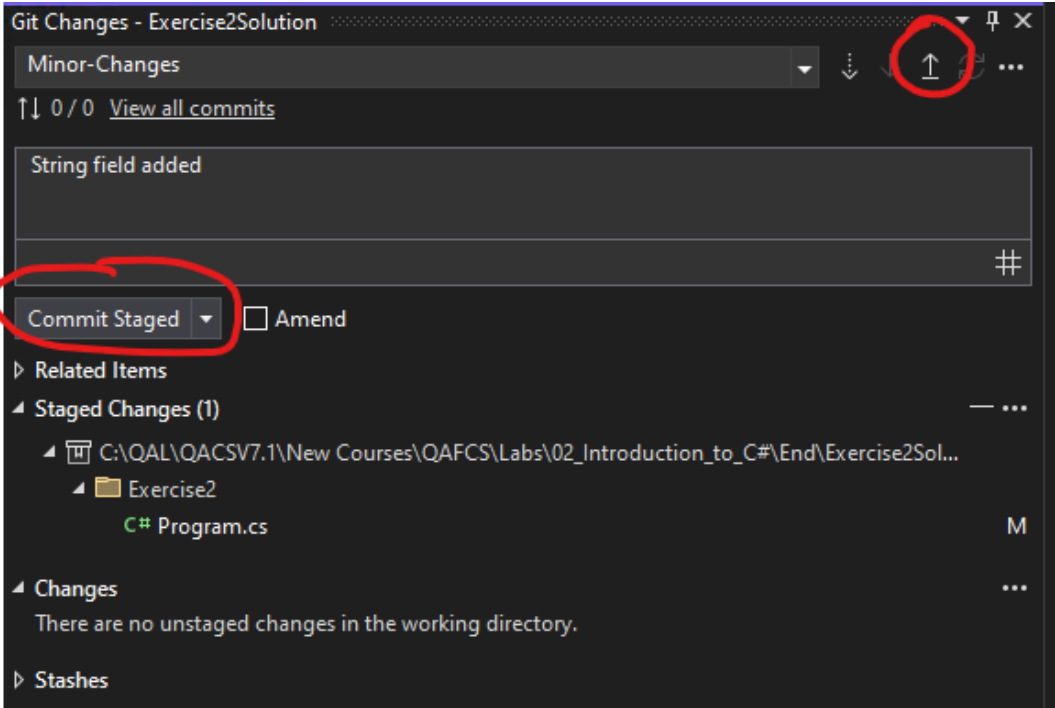
14	
15	If not already done so, open the Program.cs file and declare a private field of type string named s1 and initialise this to have a default value of "test"
16	<pre>// See https://aka.ms/new-console-template for more information Console.WriteLine("Hello, World!"); Console.WriteLine("Hello from C#"); String s1 = "test";</pre>
17	Notice that the modified file has a red tick associated with it in the Solution Explorer window

18	
19	Open the Git Changes tab and double click on the Program.cs file. The diff view appears highlighting the change (Additional line) added to the source code
20	Additionally, the status bar at displayed at the bottom of the Visual Studio window displays 1 change to the file associated with the branch named Minor-Changes
21	

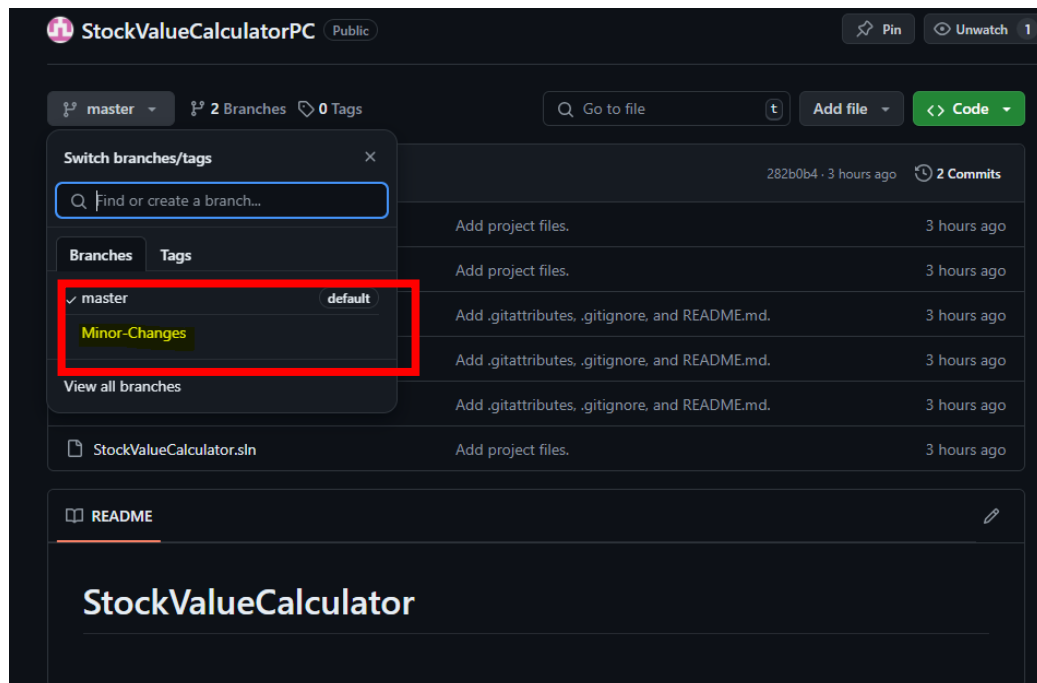
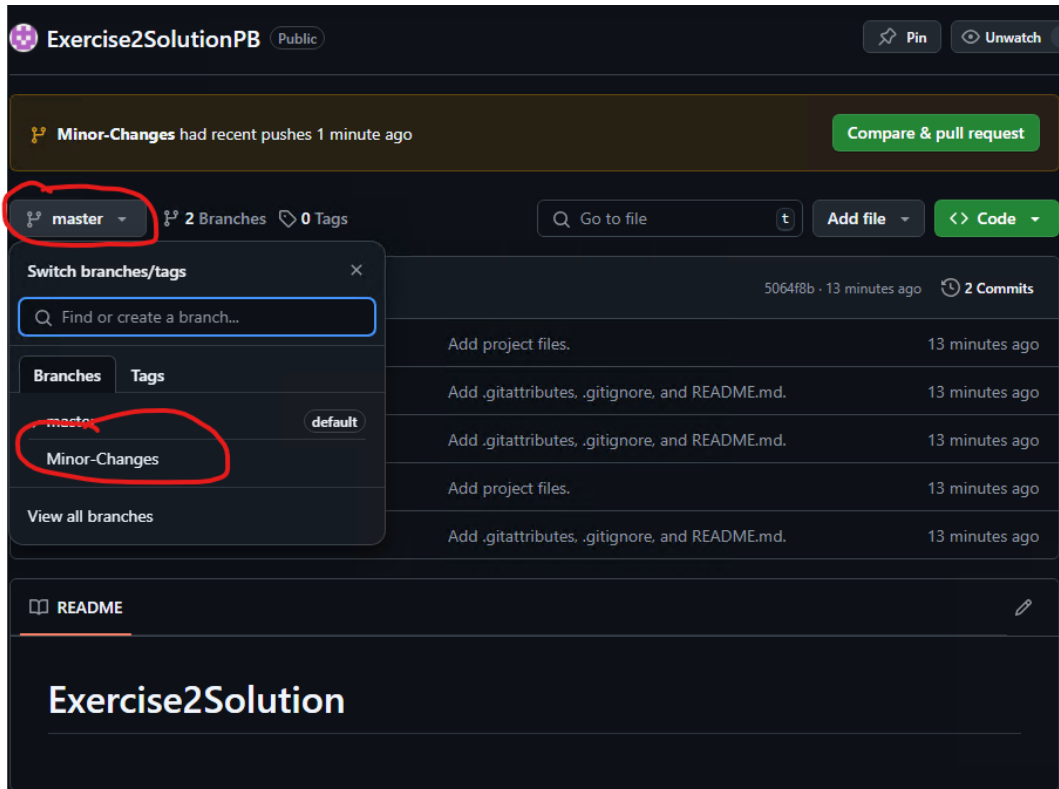
22

Now we will commit the changes to the GitHub Repo. In Solution Explorer right click on the Program.cs file select the Git menu item and then select the Commit or Stash item. Enter a description in the input text field, such as “string field added”. and click on the + Stage All button



23	The UI should now display as shown below. Click on the Commit Staged button. Then click on the up arrow to upload the changes to the Minor-Changes Branch 
24	Check that the changes have been pushed to the repo by using a browser to navigate to https://github.com/ and select the Exercise2SolutionXX repo.

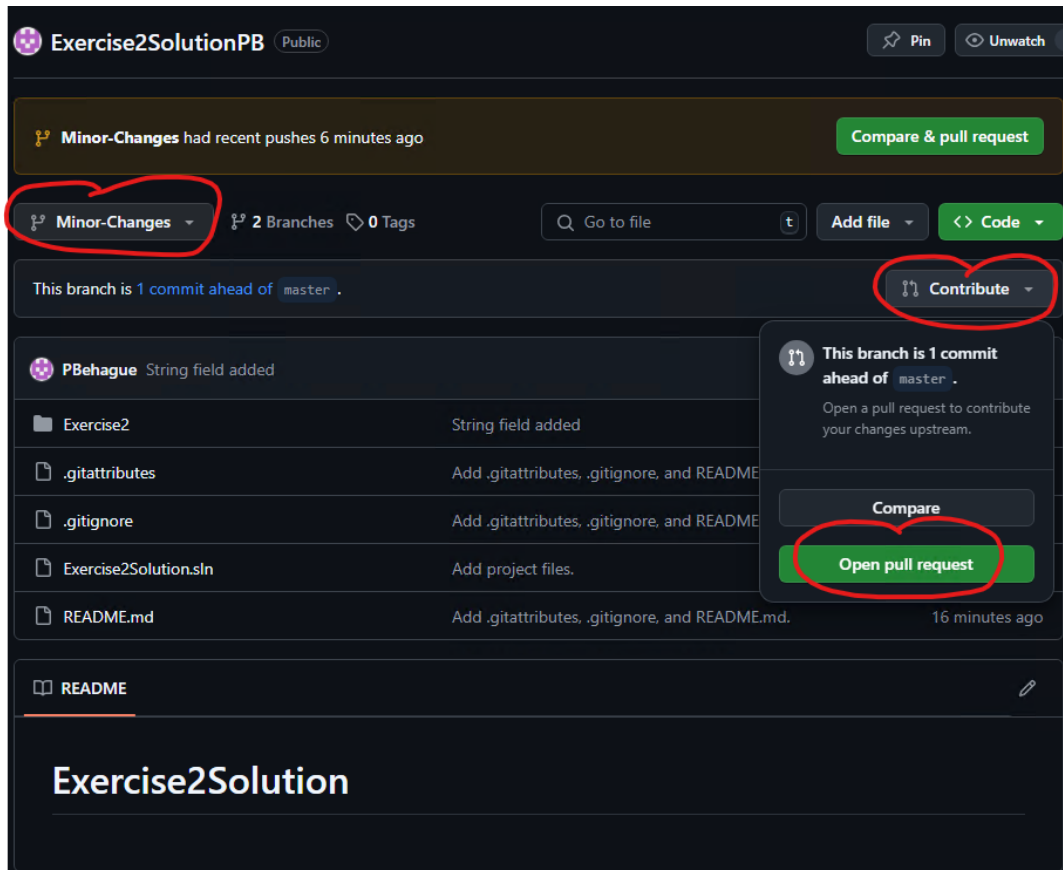
25 Navigate to the Minor-Changes branch using the drop down menu

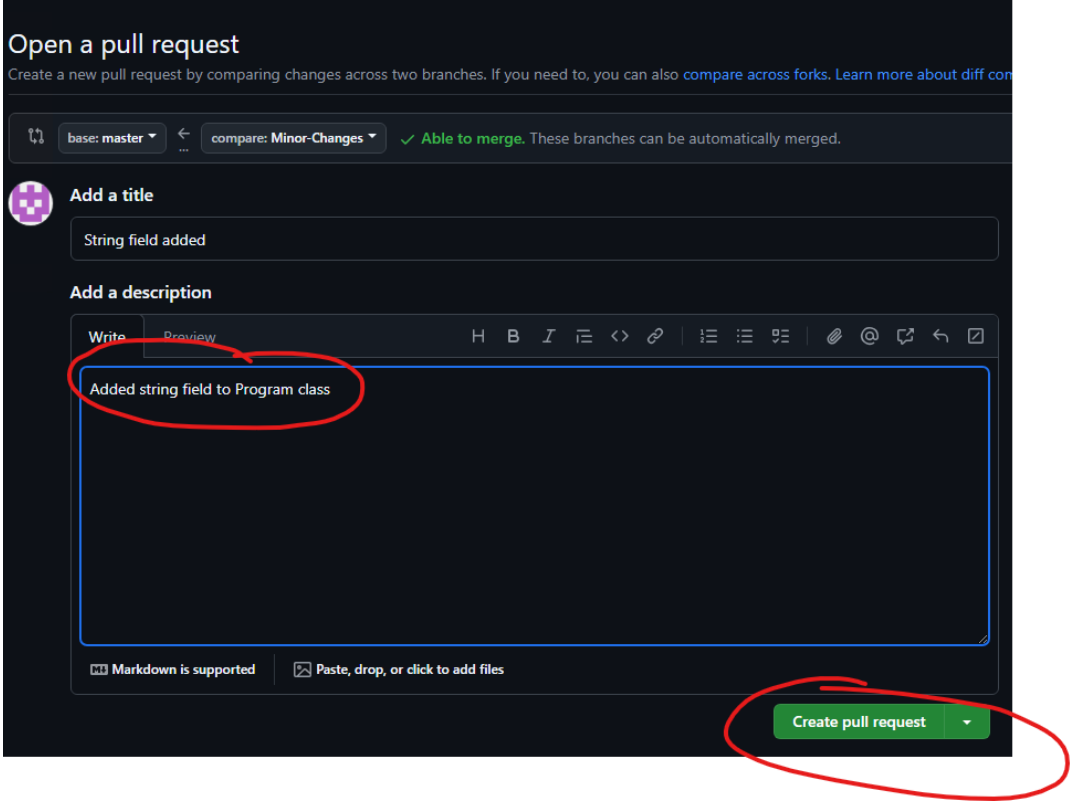


26 Open the Exercise2 folder that contains the Program.cs file. You should see the modifications have been pushed to the branch

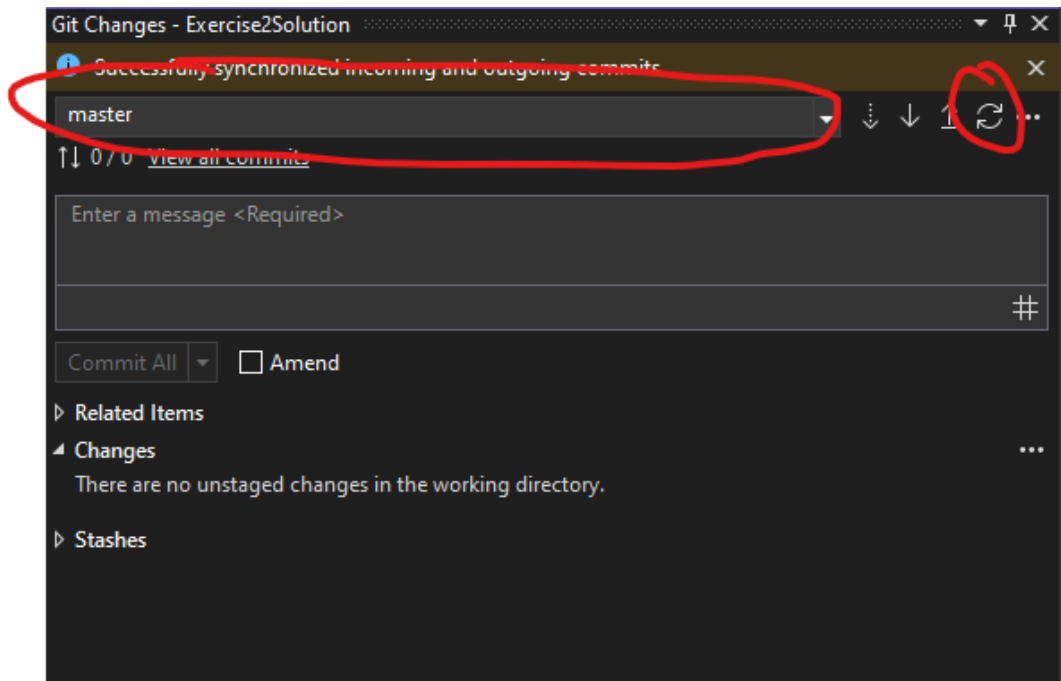
named Minor-Changes. The master branch should still have the original version. We will now merge the changes recorded in the Minor-Changes branch with the master branch

- 27 In the GitHub Portal ensure the Minor-Changes branch is selected in the dropdown. Then from the Contribute menu click **Open Pull Request**

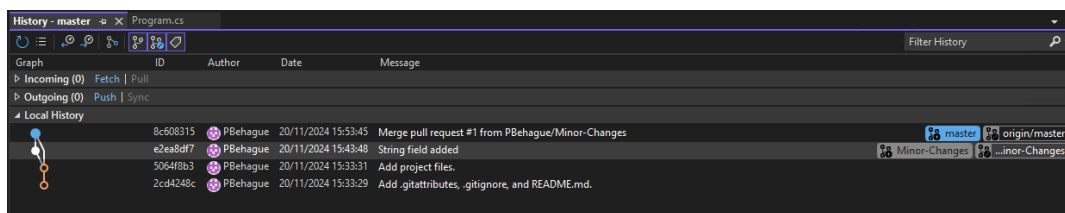


28	In the form that appears add a description e.g. Added String Field to Program class and then Click on the Create Pull Request button. 
29	On the next page click on the Merge pull request and then Confirm merge buttons in order to commit the changes with the master branch

- 30 Go back to Visual Studio select the **master** branch from the dropdown in the **Git Changes** tab and click on the **Sync (Push and Pull)** button



- 31 Go to Visual Studio's Git menu and select the View Branch History option. The graph displaying the updates to the master branch will now include the change you recorded the Minor-Changes branch.



- 32 Feel free to use GitHub when working on other exercises in the labs that follow this one.

Variables and Datatypes

The objective of this exercise is to consolidate your understanding of variables, datatypes, string formatting, and operators.

1	Open the Begin Solution for Exercise 3. C:\Courseware\QACS\Labs\03_Variables_and_Datatypes\Begin\Exercise3Solution\Exercise3.sln
2	All of the code is commented out. Go through all the numbered points (1 to 14), uncommenting each line as you go. In the case of question 10, there are a few lines to uncomment. Almost all of the lines of code have something wrong: either they won't compile or if they compile and run, they give an unexpected result. Point 14 involves setting a breakpoint to confirm the code behaves as expected. For all other code, identify what is wrong and fix it.
3	There is a Hints project if you need some tips.
4	A complete Corrected Solution is provided in the End folder for your reference.

Functions

The objectives of this practical session are twofold.

Firstly, to practice writing some simple method calls on some pre-supplied methods that have a varying number of parameters from 0-3, some of which return a value.

Secondly, to get some practice reading input that has been typed in at the Console in response to a prompt and processing that data further including converting it to a different type of data.

In this practical, you will open and code inside a couple of pre-supplied Console applications. In the first one you will follow the instructions that are inside the Program class' Main method that simply need to be followed accurately.

In the second, you will follow step by step instructions in this document.

Part 1 – Calling pre-supplied methods

1	Open the existing pre-supplied Visual Studio solution (.sln) called ' MethodCallingPractice '. It can be found in folder labs\04 Functions\ Begin\MethodCallingPractice .
2	Open file Program.cs that contains the code of a class called ' Program '. Expand the Main method and follow the 'tasks' reading carefully. Make use of the Intellisense trying to type as little as possible.
3	There is a small ' whenYouHaveTime() ' method which will enable you to find out 5 interesting facts about Chile, Russia, Mongolia, Finland & Zimbabwe.

Part 2 – Interacting with the Console, performing data conversions and authoring a helper routine

1	Open the existing pre-supplied Visual Studio solution (.sln) called ' DoingLunch '. It can be found in folder labs\04 Functions\Begin\DoingLunch .
2	Open file Program.cs that contains the code of a class called ' Program '. Expand the Main method. The scenario is going to mimic a serving line at a lunch hall in that we are going to prompt the user to answer certain questions. What would you like as a main dish? Then how many Roast Potatoes? How many Brussel Sprouts? Then display what their lunch is.
3	You will note that there are currently three prompts pre-coded with the lower down ones commented out. The first prompt is: <code>Console.WriteLine("what main dish would you like?");</code>
4	Add 2 lines of code. One that declares a string called mainCourse. Another that accepts an entry from the keyboard and stores it into the mainCourse variable. Use Console.ReadLine(). <code>string mainCourse; mainCourse = Console.ReadLine();</code>
5	Add code at the very bottom of the method that prints out a message to the user along the lines of: "Hello, your lunch is xxxxx" Where xxxx is the mainCourse the user entered.

	<pre>Console.WriteLine ("Hello, your lunch is " + mainCourse);</pre>
6	However, in view of the fact that this message is going to get significantly larger let's update it immediately using the 'special' version of Console.WriteLine() <pre>Console.WriteLine ("Hello, your lunch is {0}", mainCourse);</pre>
7	Catching Numerical Information If we want a user to input numerical information we have a slight problem in that Console.ReadLine() treats all input as a string. Consequently, we need to catch the string in a similar way to the previous steps and then convert it to a numerical value. Let's get the user to enter the number of roast potatoes they want.
8	Uncomment the "How many roast potatoes?" prompt. Immediately after this statement declare a string variable called sRoasties and store the inputted value into it.
9	We could clearly display this 'string' value in the message but later we will want to do arithmetic on it so we need to convert the string type into an int. So declare an int variable called roastCount and on the next line use the Convert.ToInt32() method in the following way: <pre>int roastCount; roastCount = Convert.ToInt32(sRoasties);</pre>
10	Uncomment the "How many roast potatoes?" prompt. Immediately after this statement declare a string variable called sRoasties and store the inputted value into it.
11	Repeat the previous three steps to retrieve and display information about the number of brussel sprouts the user would like.

- | | |
|----|---|
| 12 | The message should now look like
“Hello, your lunch is xxxxx with yy roast potatoes and zz brussel sprouts.” |
|----|---|

Creating a helper routine

Have you noticed the duplication of effort in our code?

The logic behind the requests for roasts and Brussel sprouts along with the associated numeric conversions is pretty much the same. This is the perfect opportunity to re-factor the code and create a method that does the job once but that can be called anytime we need numeric data.

The first thing we need to do is determine exactly what is common and what is unique.

Observe that when we ask for data we display unique messages that ask the user to enter either one thing or the other but the code that converts this “value” to an actual number is essentially the same.

- | | |
|----|--|
| 13 | Create a static method called <code>GetInteger</code> that takes a string called <code>message</code> as a parameter. The method's return type should be <code>int</code> . Do this immediately after the end of the <code>Main</code> method.

The parameter (if and when the method gets called) will contain the message that will be displayed to the user telling them what to enter. For example “How many roast potatoes?” This return value will be the number the user entered. |
| 14 | Add code to this method that displays the message parameter on the console, accepts a string input, converts this string to an integer and returns the result of this conversion to the caller. Words like ‘roast’ and ‘brussels’ have no meaning in this method, it has to work for all questions/answers. Your code may look like the following: |

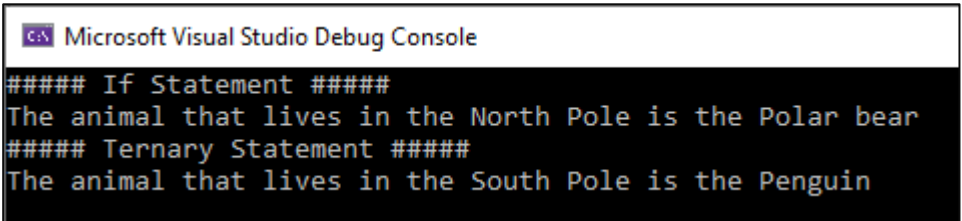
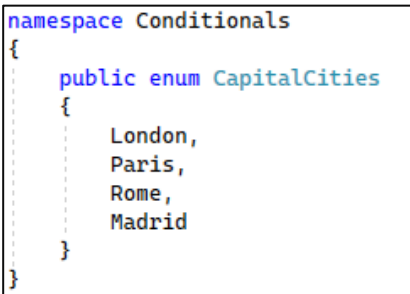
	<pre>static int GetInteger(string message) { Console.WriteLine(???); ?? ?? = Console.ReadLine(); int ?? = Convert.ToInt32(??); return ??; }</pre>
15	Can you see that this code is essentially the same as the code that retrieves the 'number of roasts' and 'number of brussels'? It's just that instead of "hard coding" the "How many roasts?" or "How many brussels?" we will pass them as parameters to the method and it will display them.
16	Replace the 3 lines of code in the main method that prompt for "how many roasts?" that catch the reply, and convert it to an integer with a one line call to the GetInteger method. You need to work this out for yourself:
17	Now in a similar way, replace the 3 lines of code that prompt for "how many brussels" and convert it to an integer with another one line call to the GetInteger method.
18	Build and test your application.
19	Can you see though that the GetInteger method only needs to be 2 lines long. The 2 nd line will start with 'return'! Re-factor it now.
20	After receiving the message of what his lunch is he sits down to eat it. Unfortunately, brussel sprouts were the only vegetable choice which is why he took some. But he remembered he really does not like brussel sprouts at all but unwilling to see food wasted – and he is hungry - he comes up with a cunning plan. If he cuts the roast potatoes into quarters. He could then perhaps 'force down' one brussel sprout with each quarter of roast potato. Fortunately, his

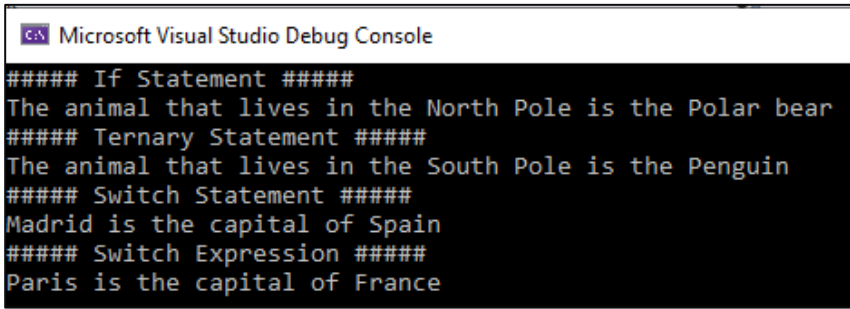
	lunch partner has a spare whole roast potato which he gratefully 'grabs'.
21	19. Write some code to work out how many spare quarters of roast potato he will have after receiving the extra one and cutting them into quarters and using up 1 per brussel sprout. Display an appropriate message. As the answer cannot be negative (in the situation where he has, say, 3 potatoes but 18 sprouts) use <code>Math.Max</code> to ensure that 0 is displayed if he runs out of quarter-roasts before he eats all the sprouts. Place this code at the bottom of the Main() method. Test it fully. Now reduce it to a single statement.

Conditionals

The objective of this exercise is to consolidate your understanding of conditional statements, including the if statement, the ternary statement, switch statements, and switch expressions.

1	Create a new console project called Conditionals in: C:\Courseware\QACS\Labs\04_Conditionals\Begin\
2	Delete the contents of Program.cs .
3	Add an enum to the project in a file called Pole.cs . <pre>namespace Conditionals { public enum Pole { North, South } }</pre>
4	In Program.cs, declare a variable called pole and assign it the value of Pole.North. Fix any issues using Visual Studio Quick Actions and Refactorings (Ctrl+dot). Create a second variable of type string, called animal. Write an if statement that tests whether the value of pole is equal to North and if true, assigns the value of 'Polar bear' to the animal variable. Otherwise, assign the value 'Penguin' to the animal variable.
5	Output a message to the console:

	Console.WriteLine(\$"The animal that lives in the {pole} Pole is the {animal}"); Run the app and confirm the logic works as expected.
6	Now assign the value of Pole.South to your pole variable and perform the same conditional test using the ternary statement.
7	Output a message to the console: Console.WriteLine(\$"The animal that lives in the {pole} Pole is the {animal}"); Run the app and confirm the logic works as expected. You should now have two outputs:  <pre> Microsoft Visual Studio Debug Console ##### If Statement ##### The animal that lives in the North Pole is the Polar bear ##### Ternary Statement ##### The animal that lives in the South Pole is the Penguin </pre>
8	You will now practise with <i>switch statements</i> and <i>switch expressions</i>. Add an enum to the project called CapitalCities:  <pre> namespace Conditionals { public enum CapitalCities { London, Paris, Rome, Madrid } } </pre>
9	In Program.cs, declare and initialise the following variables:

	<pre>Console.WriteLine("##### Switch Statement #####"); var city = CapitalCities.Madrid; string countryMessage = "";</pre>
10	Write a switch statement that switches on the city value against the four values in the enumeration. Within each block, assign a message to the countryMessage variable: <pre>countryMessage = \$"{city} is the capital of France";</pre>
11	Add a default case label and after the switch statement, output the following: <pre>Console.WriteLine(countryMessage);</pre>
12	Now see if you can achieve the same behaviour with a switch expression. Assign the value of Paris to the city variable <i>before</i> the switch expression and output a message to the console <i>after</i> the switch expression.
13	When you have completed your code, run the program. You should now have the following output:  <pre>Microsoft Visual Studio Debug Console ##### If Statement ##### The animal that lives in the North Pole is the Polar bear ##### Ternary Statement ##### The animal that lives in the South Pole is the Penguin ##### Switch Statement ##### Madrid is the capital of Spain ##### Switch Expression ##### Paris is the capital of France</pre>

14	A suggested solution is provided in the End folder for your reference.
----	---

Loops and Collections

The objective of this exercise is to consolidate your understanding of loops and collections.

Averages

1	Create a new console project called Averages in: C:\Courseware\QACS\Labs\05_Loops_and_Collections\Begin\
2	In Program.cs , delete the comment and the line of code and replace with the following code: <pre>AverageCalculator calculator = new AverageCalculator(); calculator.AveragesWithWhile(); Console.WriteLine("====="); calculator.AveragesWithDowhile(); Console.WriteLine("====="); calculator.AveragesWithFor();</pre>
3	Resolve the issues as follows: • Ctrl-dot on AverageCalculator, and select Generate class in new file • Ctrl-dot on each of the three methods and select Generate method • In each of the three generated methods in the AverageCalculator class, remove the line with the NotImplementedException
4	In the AveragesWithWhile method, put the following code:

	<pre>double total = 0.0; int count = 0; Console.Write("Enter the first number, or Q to quit: "); string input = Console.ReadLine();</pre>
5	On the following line, type 'while' then press tab twice to insert a code snippet. As the snippet is inserted, the word 'true' is highlighted. Overtyping that with: <pre>input.ToUpper() != "Q"</pre> The full method should now look like the following: <pre>internal void AveragesWithWhile() { double total = 0.0; int count = 0; Console.Write("Enter the first number, or Q to quit: "); string input = Console.ReadLine(); while (input.ToUpper() != "Q") { } }</pre>
6	Inside the while loop, put the following code: <pre>total += double.Parse(input); count++; Console.Write("Enter another number, or Q to quit: "); input = Console.ReadLine();</pre>
7	After the end of the while loop, put the following code:

	<code>Console.WriteLine(\$"The average of those numbers is {total / count}");</code>
8	Test AveragesWithWhile by running the program. Press F5, enter some numbers when prompted, and when you're ready to see the average, enter Q to quit.
9	Take a moment to look at the code you've just written. Note the following points: • In the condition for the while loop, you use input.ToUpper rather than just input. This means that the loop will end when the user types either 'q' or 'Q', because you turn the user's input to upper-case before comparing it to the letter 'Q'. • You have to read from the console immediately before the start of the while loop to decide whether to enter it for the first time. You do this again at the end of the loop to decide whether to repeat the loop. This pattern, where you set a variable before the loop and then change it at the very end of the loop, is quite common. • Note how you use double.Parse to parse the string input from the user. Console.ReadLine will only ever return a string, so if you want to treat the input data as any other data type, it will always need to be parsed first. • If the user types 'Q' in place of the first number, the loop will not execute at all. This is a key feature of while loops; they execute zero or more times. • What do you think the outputted result will be if you type 'Q' in place of the first number? Try it and see if you are right.
10	In the previous step, when you tried entered 'Q' to quit without performing any average, the program displayed the output 'NaN'. This stands for 'Not a Number', because when you divide by zero there is no valid numeric answer. (As an aside, if you divide an <i>integer</i> by zero, you will get an <i>exception</i>, but if you divide a <i>double</i> by zero the answer is <i>NaN</i> and there is no exception.)

	Let's fix that problem. Replace the final Console.WriteLine with the following: <pre>if (count == 0) { Console.WriteLine("You didn't enter any numbers"); } else { Console.WriteLine(\$"The average of those numbers is {total / count}"); }</pre>
11	Confirm the behaviour of the program by running it and immediately entering 'Q' versus running it and entering some valid numbers before quitting.
12	Now see if you can achieve the same behaviour with a do...while loop. Write code within AverageWithDoWhile. Use the 'do' code snippet to get started.
13	When you have completed your code, run the program. When you press F5, the first sequence of numbers you enter is controlled with a while loop. After you press Q, it will prompt you for a second set of numbers; this is what is being controlled by the new code that you have written using a do...while loop. Check that both types of loops give correct averages.
14	Now see if you can achieve the same behaviour with a for loop.

	Write code within AverageWithFor. Use the 'for' code snippet to get started. Because for loops run a set number of times, start off by asking the user how many numbers they have before entering the loop.
15	When you have completed your code, run the program. When you press F5, the first sequence of numbers you enter is controlled with a while loop and the second set are controlled by the do...while loop. You should then be prompted for how many numbers you have before the for loop calculates and displays the average. Check that all three loops give correct averages.
16	A suggested solution is provided in the End folder for your reference.

Collections

1	Create a new console project called Collections in: C:\Courseware\QACS\Labs\05_Loops_and_Collections\Begin\
2	Delete the contents of Program.cs.
3	Create an <i>array of strings</i> called muppets to store the following values: 'Kermit the Frog', 'Miss Piggy', 'Fozzie Bear', 'Gonzo', 'Rowlf the Dog', 'Scooter', 'Animal', 'Rizzo the Rat', 'Pepe the Prawn', 'Walter', 'Clifford'

4	Use a foreach loop to write the strings to the console.
5	Create a strongly typed <i>list of strings</i> called muppetList .
6	Add the contents of the muppets array to the list using a single method call.
7	Output the following values to the console: • The first muppet • The last muppet
8	Add a new string to the list: 'Beaker'
9	Output the value of the last muppet in the list to the console and confirm it is now 'Beaker'.
10	You will now confirm the list is strongly typed by attempting to add non-string types to the list: • Attempt to add the Boolean value <i>true</i> to the list • Attempt to add the int value <i>3</i> to the list Comment out any code that will prevent a successful build.
11	Your next task is to sort the list. Firstly, use a loop to display all the strings on one line separated by commas. Then, sort the list and re-display the strings to confirm they are sorted alphabetically.

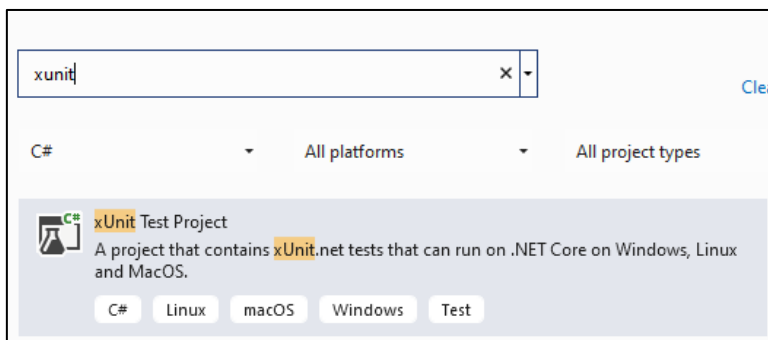
12	You will now use different operators to extract specific strings from the list. Use an <i>indexer</i>, the <i>index from end</i> and <i>range</i> operators, to extract the following strings: • The first string • The last string • The second to last string • A slice of the strings in position 5 and 6
13	You will now create a <i>dictionary</i> to store strings for the keys and values. Call this dictionary muppetdict.
14	Add the following items to the dictionary: • 'Beaker', 'Meep' • 'Miss Piggy', 'Hi-ya!' • 'Kermit', 'Hi-ho!' • 'Cookie Monster', 'Om nom nom'
15	Create a variable catchphrase and extract the value for Miss Piggy Output this to the console.
16	You will now write three loops to iterate over the KeyValue pairs, Keys, and Values respectively. Within each loop, output a string containing the iterated item.
17	A suggested solution is provided in the End folder for your reference.

Unit Testing

The objective of this exercise is to consolidate your understanding of Unit Testing in C#.

- 1 Create a new project in Visual Studio called **Exercise7Tests** in the Labs\07UnitTests\Begin folder.

In the search box type **xunit**.

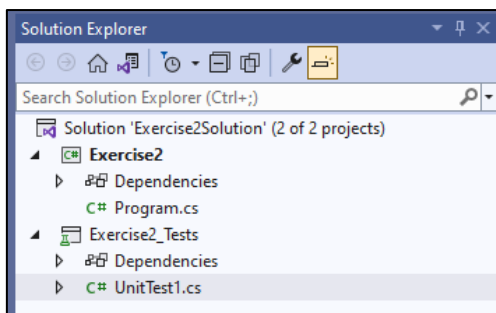


Select **xUnit Test Project** and click **Next**.

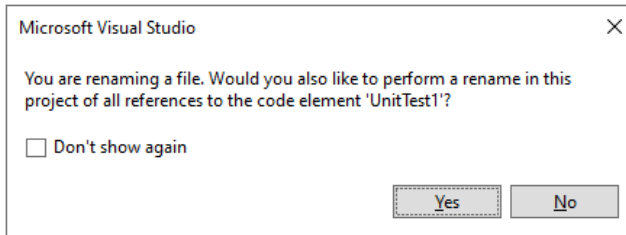
Name the project **Exercise7_Tests** and click **Next**.

Ensure **.NET 10.0 (Long-term support)** (or whatever version of .NET you are working with) is selected and click **Create**.

- 2 Your **Solution Explorer** window now contains one solution with two projects:



- 3 Rename **UnitTest1.cs** (by right-clicking on the file name) to **SimpleTests.cs** and select **YES** when the following prompt displays:

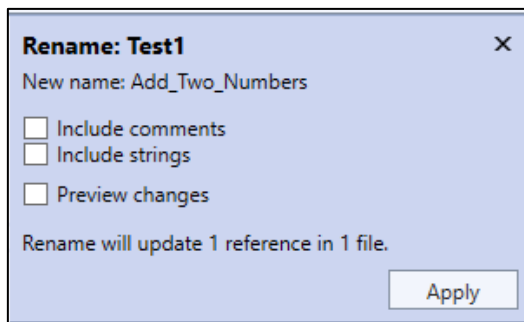
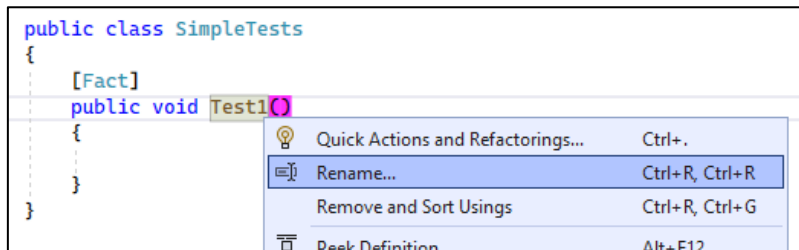


- 4 Your **SimpleTests.cs** file contains the following starter code:

```
using Xunit;

namespace Exercise2_Tests
{
    public class SimpleTests
    {
        [Fact]
        public void Test1()
        {
        }
    }
}
```

- 5 Rename **Test1** to **Add_Two_Numbers**:



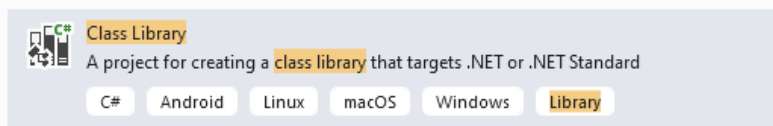
Click **Apply**.

```
using Xunit;

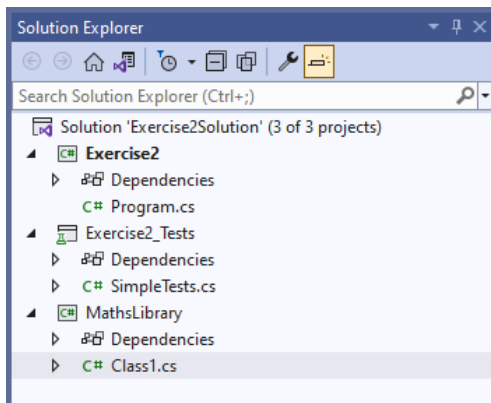
namespace Exercise2_Tests
{
    public class SimpleTests
    {
        [Fact]
        public void Add_Two_Numbers()
        {
        }
    }
}
```

- 6 You are going to write some simple tests for a Calculator. This calculator is going to be created in a new project of type Class Library.

Add a new **class library** project to the solution called **MathsLibrary**.



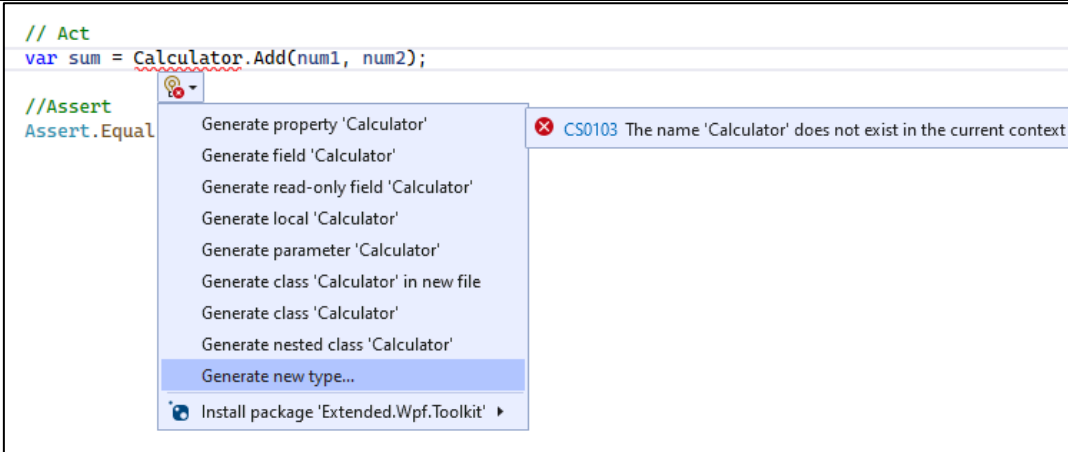
Solution Explorer should now look as follows:



- 7 Delete the file **Class1.cs**.

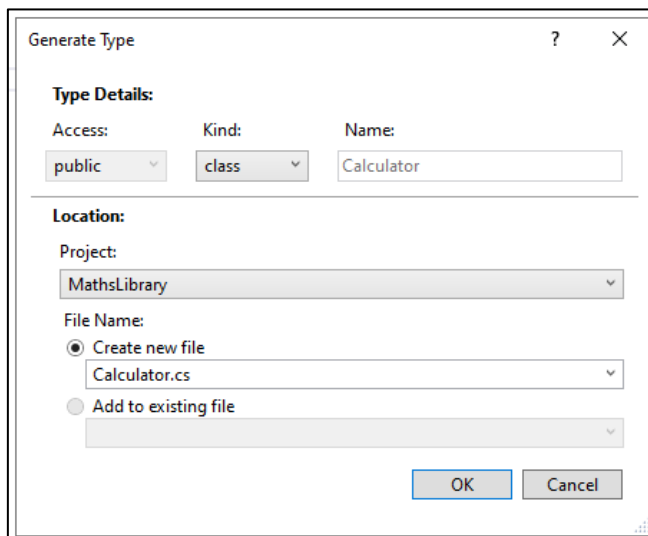
- 8 In **SimpleTests.cs** add the following code to **Add_Two_Numbers**:

	<pre> [Fact] public void Add_Two_Numbers() { // Arrange var num1 = 5; var num2 = 2; var expectedValue = 7; // Act var sum = Calculator.Add(num1, num2); //Assert Assert.Equal(expectedValue, sum); } </pre>
9	This test code uses the standard testing pattern called the triple A pattern: <i>Arrange, Act, Assert</i>. <i>Arrange</i> is for setting up items you need for the test. <i>Act</i> is for carrying out the action you are testing. <i>Assert</i> is for confirming the acted upon code behaves as expected.
10	The arrange phase creates three variables: num1, num2, and expectedValue. The act phase calls an Add method on a Calculator, passing in num1 and num2 as parameters and assigning the result to a variable called sum. The assert phase checks whether the expectedValue and the sum values are equal. The Calculator type does not exist so you will use Visual Studio to help you create it. Press Ctrl+. (Ctrl+dot) on Calculator to see the available options:



You want Calculator to be created in your **MathsLibrary** project rather than locally within the Test project so choose '**Generate new type...**'

- 11 In the dialog box, ensure a **public class** will be created and change the project to **MathsLibrary** and **Create new file**:

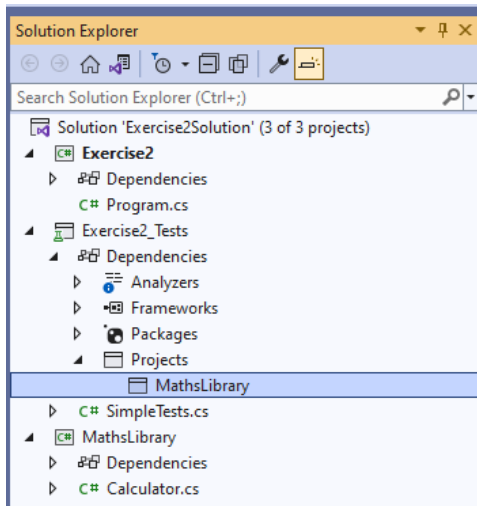


Click **OK**.

- 12 Visual Studio has created a new class (a kind of type) called **Calculator** in **MathsLibrary**:

```
1 namespace MathsLibrary
2 {
3     public class Calculator
4     {
5     }
6 }
```

Visual Studio has also added a *reference* to the **MathsLibrary** project:



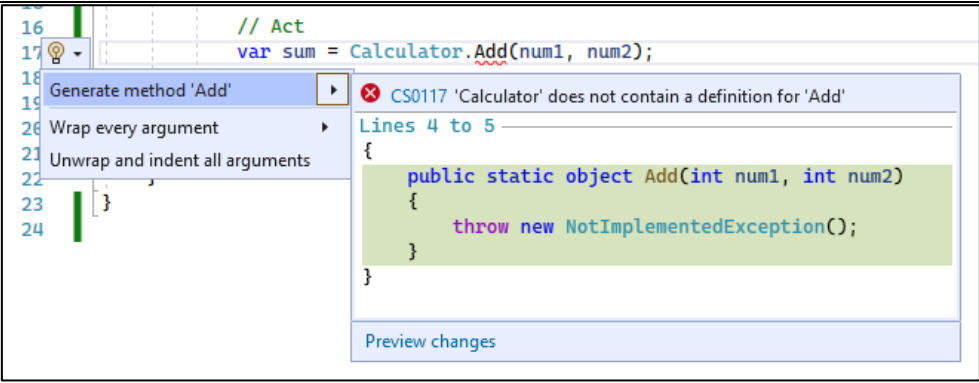
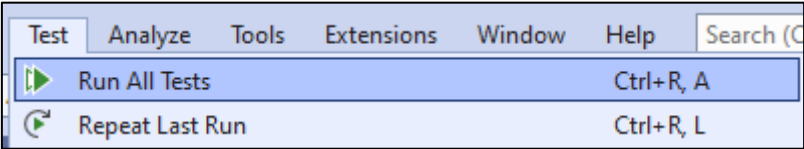
It has also imported the **MathsLibrary** namespace into your test project:

```
SimpleTests.cs
Exercise2_Tests
1 using MathsLibrary;
2 using Xunit;
3
```

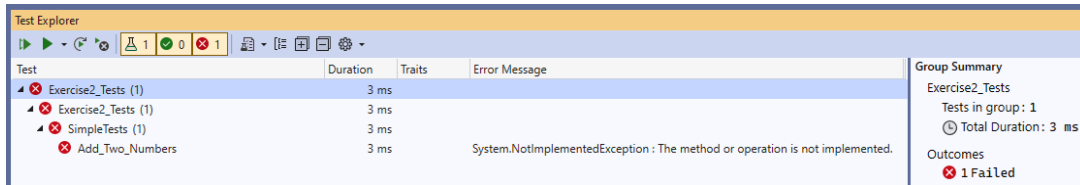
- 13 The act phase of the test code now recognises the **Calculator** type but displays an error because **Calculator** does not contain a definition for **Add**:

```
// Act
var sum = Calculator.Add(num1, num2);
```

- 14 Use **Ctrl+dot** on **Add** to generate the method:

	
15	Calculator.cs now contains an Add method: <pre>public class Calculator { public static object Add(int num1, int num2) { throw new NotImplementedException(); } }</pre>
16	You want your Add method to return whole numbers so change the word object to int. <pre>public static int Add(int num1, int num2) { throw new NotImplementedException(); }</pre>
17	You will run the test and observe the outcome. Test -> Run All Tests. 
18	Ensure the Test Explorer window is visible: Test -> Test Explorer.

Expand the Test until you see the **Add_Two_Numbers** failed test (it appears in red) alongside the error message: **The method operation is not implemented.**



- 19 You will edit the **Add** method code to ensure the method is implemented and confirm that the test passes.

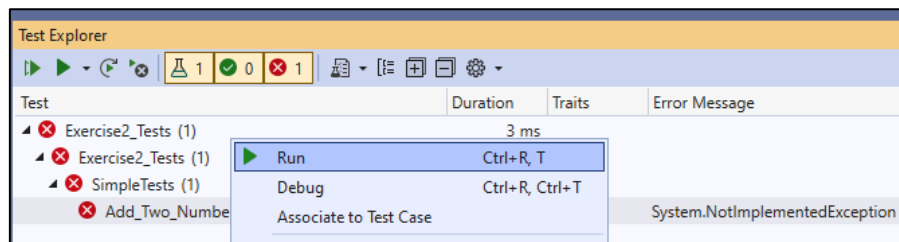
Delete the line of code: **throw new NotImplementedException;**

Replace the code with: **return 7;**

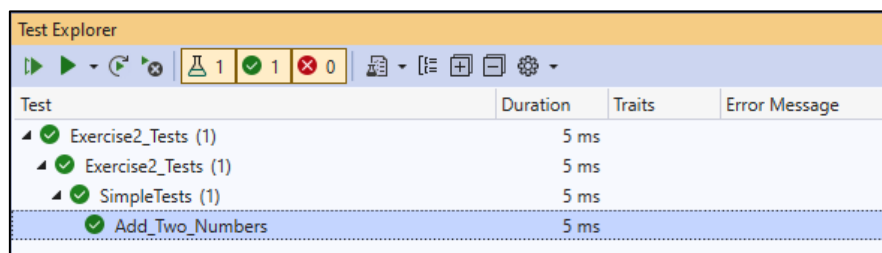
```
public static int Add(int num1, int num2)
{
    return 7;
}
```

This is hard-coding the expected value, which allows you to confirm the test is working correctly.

- 20 Right-click the failed test in **Test Explorer** and select **Run**.



The test should now pass:



21	The final step is to refactor the code within the method to perform the calculation so that additional tests adding different integers will also pass. Edit the Add method as follows: <pre>public static int Add(int num1, int num2) { return num1 + num2; }</pre>
22	Re-run the test to ensure it continues to pass. The process that you just followed is called Test-Driven Development (TDD). It follows a three-stage approach referred to as <i>red-green-refactor</i>, whereby you write a test before implementing the code. You ensure the test fails. This is the red stage. This is to guard against any false positives. You then write enough implementation to get the test to pass. This is the green stage. You then refactor the code to improve the implementation, ensuring the tests still pass.
23	If you have time, write a test for a Subtract method, then use Visual Studio to help build the implementation. Use Test Explorer to run your tests.
24	Solutions are provided in the End folder for your reference.

If you have time

25	Open the IfYouHaveTime solution located in Labs\07_unit_Testing\Begin folder.
26	Look at the code in the Program.cs file and notice it contains a reworking of the code you wrote for the 04 Functions lab. The code that determines which animal lives at a specified Pole has been refactored to live in a function called GetArcticAnimal and the code that determines which country a capital city is located in has also

	been refactored into a different function called GetCountryForCapitalCity
27	Add an xUnit Test project to the solution and create sets of tests that comprehensively prove both the GetArcticAnimal and GetCountryForCapitalCity functions work correctly. You may find it problematical to create a test that ensures the correct message for a capital city that does not exist in the CapitalCities enumeration gets returned by the GetCountryForCapitalCity function. Note, it is possible to cast an integer that has no equivalent value in an enumeration to be of that type: <pre>capitalCities city = (CapitalCities)99;</pre>

Object-Oriented Programming (OOP) Concepts

The objective of these exercises is to consolidate your understanding of object-oriented programming (OOP) concepts and to experience the run time behaviours of reference and value types. The second exercise aims to start authoring classes with instance methods and data.

Part 1 – Calling pre-supplied methods

1	Open the existing pre-supplied Visual Studio solution (.sln) called 'MethodCallingPractice'. It can be found in folder <code>labs\08 OOP\Begin\Types</code> .
2	Open the file Program.cs . Inside the Main method you will see two commented out lines that make calls to a couple of test methods. The 2 test methods contain code that creates and manipulates objects of type <code>Account</code> (a reference type) and <code>int</code> (a value type).
3	Open the file Account.cs. You will see the implementation of a reference type (a class) that represents a bank account object. In Program.cs, expand the "Testing Ref Type behaviour" region and examine the TestRefType method and the two PassAccountByxxxx methods. You will note that the balances of the 2 accounts are being displayed 4 times. Create a table like the one below, either with a pen and paper or in NoteBook.exe and by just reading the code, write down what you

	think the Balance will be when read from the variables ac1 and ac2 at these four points in the code. <table> <tr> <td></td><td>ac1.Balance</td></tr> <tr> <td>ac2.Balance</td><td></td></tr> <tr> <td>1st</td><td>_____</td></tr> <tr> <td>_____</td><td></td></tr> <tr> <td>2nd</td><td>_____</td></tr> <tr> <td>_____</td><td></td></tr> <tr> <td>3rd</td><td>_____</td></tr> <tr> <td>_____</td><td></td></tr> <tr> <td>4th</td><td>_____</td></tr> <tr> <td>_____</td><td></td></tr> </table>		ac1.Balance	ac2.Balance		1st	_____	_____		2nd	_____	_____		3rd	_____	_____		4th	_____	_____	
	ac1.Balance																				
ac2.Balance																					
1st	_____																				

2nd	_____																				

3rd	_____																				

4th	_____																				

4	Uncomment the line in the Main function that calls the TestRefType method and run the program checking how many of the values you got correct.																				
5	If you answers differed from those shown, use the Visual Studio debugger to step through the code. If you are unsure about the results, please ask your instructor to explain them.																				

Working with value types

In this section of the lab, you will see how value types work, and how they differ in their behaviour from reference types.

6	In C# the int data type (alias for System.Int32) is not a class it is a value type and thus will exhibit value type behaviour.															
7	Comment out the call to TestRefType in the Main method of the Program class.															
8	Uncomment the call to TestValueType in the Main method of the Program class.															
9	Examine the method TestValueType and the two PassIntByxxxx methods. You will note that the ints are being displayed 4 times. Just by reading the code, write down in the table below what you think the value will be when read from the variables num1 and num2 at these four points in the code. <table><thead><tr><th></th><th>num1</th><th>num2</th></tr></thead><tbody><tr><td>1st</td><td>_____</td><td>_____</td></tr><tr><td>2nd</td><td>_____</td><td>_____</td></tr><tr><td>3rd</td><td>_____</td><td>_____</td></tr><tr><td>4th</td><td>_____</td><td>_____</td></tr></tbody></table>		num1	num2	1st	_____	_____	2nd	_____	_____	3rd	_____	_____	4th	_____	_____
	num1	num2														
1st	_____	_____														
2nd	_____	_____														
3rd	_____	_____														
4th	_____	_____														
10	Run the program and check how many of the values you got correct. If you answers differed from those shown, use the Visual Studio debugger to step through the code. If you are unsure about the results, please ask your instructor to explain them															
11	Now comment out the calls to TestRefType and TestValueType and uncomment the calls to TestIntArray and TestAccountArray .															

12	Read the code being executed by each method, there are 2 simple questions to be answered, choose answers. Run the code and see if you were correct.
----	---

Part 2 – Creating a simple Account class

In this next exercise you will simultaneously author class Account whilst writing code in class Program that uses your new Account data type and the functionality it exposes.

1	Open the pre-supplied solution ClientExe.sln . You will find it in Labs\08_Object_Oriented_Programming_Concepts\Begin.
2	Open class Program. Expand the Main method. Type 'Acc' – there is no data type (class) called Account (yet).
3	Right click Solution ' ClientExe ' in SolutionExplorer and choose Add/New Project. In the 'Add New Project' dialog choose 'Class Library' name it 'Finance' and ensure it is placed in the correct \Begin folder.
4	In the project 'Finance' you now have a file called Class1.cs containing the code of a class called 'Class1'. Rename the FILE in Solution Explorer to Account.cs and you will be prompted to rename the class as well.
5	You now have public class Account { }.
6	Inside Main type 'Acc' again – still no sign of data type Account.

7	In the ClientExe area of Solution Explorer right-click 'Dependencies. Choose 'Add Project Reference', from the Projects tab choose 'Finance' and click 'OK'.
8	Inside Main type 'Acc' again – still no sign of data type Account. But if you type 'Finance'<dot> you can see the Finance.Account data type. Add a 'using Finance' clause at the top of the file and you can now type 'Acc' and see that 'Account' is a known data type.
9	Inside Main declare 2 variables of type Account <pre>Account ac1, ac2;</pre>
10	Now type these 2 statements which show 2 different but useless objects being created. <pre>ac1 = new Account(); ac2 = new Account();</pre>
11	Type a 4th statement and run your code and see that the result is false. <pre>Console.WriteLine(ac1 == ac2); // these are DIFFERENT objs</pre>
12	Keep the 1st statement and DELETE the other 3.
13	You are going to add some functionality to the Account class. Declare 3 private fields inside the class noting the lowercase. <pre>private string holder; private decimal balance; // will default to 0.00 private string accNo; // can contain alphanumeric chars</pre>
14	Use the 'ctor' code snippet to author a 'default constructor'. A constructor is a special method that runs when an object is being instantiated from the class. Constructor methods can be configured to take parameters (just like ordinary functions). We will look at constructors more fully in the next session.

	Give it 2 parameters (name and balance) and 2 statements. The statements should store the parameters in the instance variables (giving the object some initial state) you will need to use the word 'this' once: <pre>public Account(string name, decimal balance) { holder = name; this.balance = balance; }</pre>
15	Returning to Main, the 2 statements that create the instances of the Account class will be showing syntax errors. This is because the constructors each need to be given a person's name and a starting balance. Edit the code as shown below. <pre>ac1 = new Account("Fred", 100); ac2 = new Account("Susy", 200);</pre>
16	Each account now has its own 'holder' and 'balance'. Each account also has its own 'accNo' but it has defaulted to null for each of them. We would like these 'Student Accounts' to each have an 'accNo' in the format 'SA-nnnn' where nnnn is an 'auto-incremented' sequential number padded to 4 digits with zeros. i.e. SA-0001, SA-0002 for Fred and Susy respectively. Declare a 4th field in Account as follows <pre>private int nxtAccNo; // defaults to 0</pre> Add this next statement to the bottom of the constructor method. Will this work do you think? Will the account numbers be SA-0001 and SA-0002 for 'Fred' and 'Susy' or will they both have SA-0001? <pre>accNo = "SA-" + (++nxtAccNo).ToString().PadLeft(4, '0');</pre> Well, they will both be SA-0001 because each account object will get its own version of nxtAccNo starting at zero.

	Solve the problem now by marking <code>nxtAccNo</code> as static this means there is only one copy of this numeric field shared between all the instances (but each instance will have its own <code>accNo</code> in format 'SA-nnnn') <pre>private static int nxtAccNo; // defaults to 0 and is shared</pre>
17	Let's prove that it has worked. Open class <code>Account</code> and author a method as follows (it won't compile yet) <pre>public string GetDetails() { }</pre>
18	We would like this method to return a tab-separated string containing the account number, the holder's name and the account balance. Easy to do via concatenation, but here is a perfect opportunity to see again how string interpolation works. Insert this single statement into the method. <pre>return \$"{accNo}\t{holder}\t{balance}";</pre>
19	Return to <code>Main</code> and after creating the 2 <code>Account</code> objects display the details of both objects on the <code>Console</code>. You really should not need to look at the code fragment below to see how to do this. <pre>Console.WriteLine(ac1.GetDetails()); Console.WriteLine(ac2.GetDetails());</pre> To achieve this you could have typed no more than 'cw' Tab Tab an 'a' a <dot> a 'g' then '()'. If you typed more than that then you may want to try it again. Run the code. You should now be seeing output showing 'Fred' and 'Susy's correct account details with unique account numbers SA-0001 and SA-0002.

20	Now add further functionality to the Account class as follows <pre>public void Deposit(decimal amt) { balance += amt; } public bool withdraw(decimal amt) { bool result = false; return result; // just to make it compile }</pre>
21	The Deposit method is straightforward – it will work, there is no upper balance. Withdraw however is a bool method as very soon you will want to ‘listen’ to see if a withdrawal has been successful. Here is the algorithm. If the amount that is being withdrawn is less than or equal to the balance on the account then the balance should be adjusted downwards and result set to true. Implement that code now. <pre>public bool withdraw(decimal amt) { bool result = false; if (?? ? ??) { ?? ? ? // change the balance ?? ? ? // ensure method returns true } return result; // to make it compile }</pre> This would work fine if there was no concept of an overdraft. But these are ‘Student Accounts’ – there is a £500 overdraft limit and it is the same for all Accounts so this should be held in a static (belonging to the class) variable. Declare a static decimal overdraftLimit with a value of 500. Also adjust the Withdraw(decimal amt) method to allow a student to withdraw as long as they do not go beyond their overdraft limit as opposed to below 0.

22	Perform a Deposit into 'ac1' and a successful Withdraw from 'ac2'. Make these method calls before the display of the details. Check the methods have produced the correct values. Now change the parameter passed to Withdraw() so that it would take the student beyond their overdraft limit and check the balance does not change. You probably coded your Withdraw method as follows? <pre>public bool withdraw(decimal amt) { bool result = false; if (amt <= (balance + overdraftLimit)) { balance -= amt; // change the balance result = true; // ensure method returns true } return result; }</pre>
23	It works fine but it is not obvious that overdraftLimit is 'shared' between all instances. Not clear that it belongs to 'Account' rather than 'this'. Make the statement clearer perhaps by changing it to. <pre>if (amt <= (this.balance + Account.overdraftLimit)) {</pre>
24	Now we wish to enable transfers between accounts so that one student could do a little internet banking and 'loan' money to a friend. It could be written as an instance method and if it was its signature might be <pre>public bool Transfer(decimal amt, Account to) { ... calls to this.withdraw and to.Deposit</pre>

	<pre>}</pre> The code that would invoke it might look like this: <pre>ac1.Transfer(100M, ac2); // 100 from ac1 to ac2</pre> Alternatively, it could be written as a class (static) method with this signature. <pre>public static bool Transfer(Account from, Account to, decimal amt) { }</pre> This latter version is perhaps more 'appropriate' for a method which by definition affects 2 accounts, but it is a matter of personal preference. Write this method signature now, it will not compile yet of course.
25	Now for the implementation. Because it is boolean it should declare a boolean result variable and set it to false (just being pessimistic). The final statement should return the result. In the main body of the method you should do a Withdraw from the 'from' account and only if that is a successful Withdraw() should it then do a Deposit() into the 'to' Account. After the Withdraw & Deposit (or neither) operations have completed you should always display the result of attempting the transfer in the format "Transfer Successful: Yes"

or

“Transfer Successful: No”

You should be able to implement that very easily without looking at the code block below that shows you the final code.

Have an attempt now. It is a good chance to test yourself out on whether you understood how the conditional operator (?:) works as you should use it to produce the “Yes” or “No” strings.

```
public static bool Transfer(Account from, Account to,
                           decimal amt) {
    bool result = false;
    if (from.Withdraw(amt))
    {
        to.Deposit(amt);
        result = true;
    }

    Console.WriteLine(
        $"Transfer Successful: {(result ? "YES" : "NO")}");
    return result;
}
```

- 26 Now go back to Main() and perform a Transfer of a modest sum from ‘ac1’ to ‘ac2’ or the other way round, depends which student ‘Fred’ or ‘Susy’ you think has blown their ‘terms’ allowance during ‘freshers week’.

Remember the method you are calling is static not instance.

- 27 Before we wrap up note that although we can display the ‘Details’ of any account object we do not have the ability to simply find out the balance on the account or in fact the name of the holder of an account.

Author and code now a simple public decimal GetBalance() method and a public string GetHolder() method. They will be needed in the challenges that follow and in the next chapter's lab.

If You Have Time: Coding Challenge 1

- 28 Add the 2 account references 'ac1' and 'ac2' to a List of Account objects, add a 3rd and a 4th if you want.
- Author a method (in Program) called ProcessAccounts with the following signature.
- ```
public static void ProcessAccounts(List<Account> accs) {
}
```
- This method should loop through the accounts it receives and add a £10 bonus (call Deposit()) to each of them.
- Call the method from Main and after getting control back loop through the array yourself displaying the account holders name and the account balance to check that they have all gone up by £10.

## If You Have Time: Coding Challenge 2

- 29 Declare and create a List of string called names exactly as follows
- ```
List<string> names = new List<string>
{"Ann", "Anne", "Annie", "Anneka", "Annabel"};
```

- 30 Declare and create a List of Account objects called studentAccs

	Declare and create an instance of class Random, it has a useful Next() method that you soon will use repeatedly.
31	Write a simple for loop that runs 'the right number of times' that creates an account for each of the names and adds the object to the studentAccs collection. The account holders name should be the 'next' name from the names array. Use instance method Next() of the Random object you created earlier to generate a random number of £ in the range 10-99 inclusive to be used as opening balance. <pre>for(int i = 0; i < ???; i++) { ???.??(new ??(??,??)); }</pre>
32	Write a foreach loop that steps through the list of accounts displaying the details of each account. Run the code. The 5 names should now have an account each with a balance in the range £10 - 99 inclusive. Now you are going to write code in a loop so that a Transfer happens from each account holder to the 'next' (5 transfers in total). i.e. it will transfer money from 'Ann's account to 'Anne's account.

From 'Anne's account to 'Annie's account etc and lastly (the tricky bit) from the final person 'Annabel' to the first person 'Ann's account.

The amount of money that each person transfers out is the length of their own name.

It is easy to write code to determine the length of the account holders name as class String has a 'Length' property.

So Ann will give £3 to Anne. Anne will give £4 to Annie. Annie will give £5 to Anneka... finally Annabel will give £7 to Ann.

Each person's balance will drop by £1 except Ann whose balance will go up by £4 (£3 out and £7 in).

Write this code now without hard coding any names or amounts.

If You Still Have Time: Coding Challenge 3

Add an xUnit test project to the solution and create a series of tests that ensure the code in the Account class is comprehensively behaving as expected.

Properties and Constructors

The objective of this exercise is to consolidate your understanding of C# properties and constructors.

The Car Library and Console Projects

1	Create a new Class Library project called CarLibrary . Rename Class1.cs to Car.cs
2	Add a Console Application project to the Solution called CarConsole . Set CarConsole as the start-up project. In CarConsole , add a project reference to the CarLibrary project.
3	In Car.cs , create a property of type <i>int</i> called Speed with a <i>backing field</i> speed . Validate that the speed set is above zero but under 100.
4	Add an auto-implemented property of type <i>string</i> called RegistrationNumber .
5	Add a calculated expression bodied property called SpeedInKilometres of type <i>double</i> . To calculate the speed in kilometres, multiply the speed by 1.609344
6	Add string properties for Make , Model , and Colour .
7	In CarConsole , in Program.cs :

	Delete the line of code that outputs 'Hello, World!' Instantiate a car object, c1. Issue a using directive to bring the CarLibrary namespace into scope. Write the name of the instance to the console: <pre>Console.WriteLine(nameof(c1));</pre> Build and run the console application to confirm the object can be successfully instantiated. Set the make of c1 to be 'Ford'. Write the <i>make</i> of c1 to the console. Write the <i>model</i> of c1 to the console. What value is displayed?
8	In the Car class, create a constructor that accepts a <i>make</i> and a <i>model</i> only. Initialise these values within the constructor.
9	In CarConsole: Re-run the app. Does it build successfully?
10	Create a <i>parameterless</i> constructor Set the make and model to be Unknown and the colour to be Black. Confirm the console app builds and runs successfully.

	What value is displayed for the model?
11	In CarConsole: Instantiate a car object, c2, using the overloaded constructor. The make is Audi, the model is TT; Write the make and model of c2 to the console. Set the colour property to Red. Write c2's colour property to the console. Set the speed of c2 to 30 miles per hour. Display the speed in the console in both <i>miles per hour</i> and <i>kilometres per hour</i>.
12	In CarConsole: Instantiate a car object, c3, using the overloaded constructor (BMW, X5) and an object initialiser that sets the colour to Grey and the registration number to ABC 123. Write the property values of c3 to the console.
13	In Car.cs, chain the parameterless constructor to the overloaded constructor, passing Unknown Make and Unknown Model as the parameters. In the body of the parameterless constructor, remove the make and model and set the colour to be White. Confirm the console application still builds and runs successfully.

14 In **CarConsole**:

Instantiate a car object, **c4**, using the parameterless constructor.

Write the property values of **c4** to the console.

Confirm **c4** is an unknown make and model that is white with an empty registration number.

If You Have Time - Coding Challenge: Enhance the Finance ClientExe Projects

1 Either return to your solution to the previous exercise (08 OOP Concepts) or open the one in the Begin folder.

2 Rework the Account class so:

- The holder field is private with access being given to it via a public property called Holder. The property should be both Gettable and Settable with the following validation being applied:
 - If the passed in value is null the value of holder should be set to "*** Anon ***".
 - If the passed in value is **not** more than 2 character the value of holder should be set to the passed in value surrounded by 3 sets of asterixes. E.G. If the passed in value is "Mo" the value of holder should be set to "*** Mo ***".
- The private balance filed should be replaced by a gettable but **not** settable property called Balance.

You will need to rework other elements of code in both the Account and Program classes in order to make the code compile.

Add code to the Program class that fully tests the new features.

- | | |
|---|--|
| 3 | <p>We want to be able to create an account object allowing the balance to default to 0. i.e. a single parameter constructor of string name only. Give the Account class this as a second constructor. Use constructor chaining (see relevant slides in the PowerPoint appendix) so the single parameter constructor calls the two-parameter constructor passing the name and a zero for the balance.</p> <p>Add code to the Program class that fully tests the new features.</p> |
|---|--|

If You Still Have Time

Add an xUnit Test project to the solution and create a set of unit tests that ensure the code in the Account class is working correctly.



QA.com